

# **Resolución De Problemas Aplicando El Modelo Relacional, Cardinalidad Y**

## **Normalización**

**GA6-220501096-AA1-EV01**

### **Autor:**

- Yeison Adolfo Díaz Tapasco

### **Institución:**

Servicio Nacional de Aprendizaje (SENA)

Programa: Análisis y Desarrollo de Software

### **Profesor:**

Oscar Fernando Carvajal

Marzo 2026

## **TABLA DE CONTENDO**

- 1. Introducción**
- 2. Objetivo**
- 3. Análisis del modelo a desarrollar**
- 4. Modelo Entidad–Relación**
  - 4.1 Entidades Identificadas**
  - 4.2 Relaciones y Cardinalidad**
  - 4.3 Diagrama Entidad – Relación**
- 5. Normalización**
  - 5.1 Primera Forma Normal (1FN)**
  - 5.2 Segunda Forma Normal (2FN)**
  - 5.3 Tercera Forma Normal (3FN)**
- 6. Modelo Relacional Final**
- 7. Dependencias Funcionales Formales**
  - 7.1 Definición**
  - 7.2 Dependencias Funcionales por Tabla**
  - 7.3 Verificación Formal de 3FN**
- 8. Script SQL Sistema de Parqueadero**
- 9. Componentes Avanzados de Base de Datos**
  - 9.1 Índices**
  - 9.2 Vistas**
  - 9.3 Funciones**
  - 9.4 Triggers**
- 10. Conclusiones**

## **1. INTRODUCCIÓN**

El presente documento desarrolla el modelo relacional de un sistema de gestión de parqueadero aplicando los principios del modelo entidad–relación, cardinalidad y normalización hasta Tercera Forma Normal (3FN).

Se busca estructurar la base de datos garantizando integridad referencial, eliminación de redundancias y prevención de anomalías de inserción, actualización y eliminación.

## **2. OBJETIVO**

Diseñar el modelo entidad–relación y transformarlo en modelo relacional cumpliendo las reglas de normalización, identificando correctamente cardinalidades y asegurando consistencia estructural en la base de datos.

### 3. ANÁLISIS DEL MODELO A DESARROLLAR

El software a desarrollar corresponde a un sistema de gestión de parqueadero que debe permitir:

- Registrar clientes
- Registrar vehículos
- Asignar espacios
- Controlar entradas y salidas
- Generar facturación
- Registrar empleado responsable

Problemas comunes que deben evitarse:

- Duplicación de datos del cliente
- Espacios ocupados por múltiples vehículos simultáneamente
- Facturas sin registro asociado
- Dependencias parciales o transitivas

Decisiones de diseño:

- Separar Cliente de Vehículo (1:N)
- Crear entidad intermedia Registro para controlar entradas
- Factura depende exclusivamente del Registro
- Separar Cargo de Empleado para cumplir 3FN estricta

## **4. MODELO ENTIDAD – RELACIÓN**

### **4.1 Entidades Identificadas**

- Cliente
- Vehículo
- Espacio\_Parqueo
- Empleado
- Cargo
- Registro
- Factura

### **4.2 Relaciones y Cardinalidad**

Cliente 1:N Vehículo

Vehículo 1:N Registro

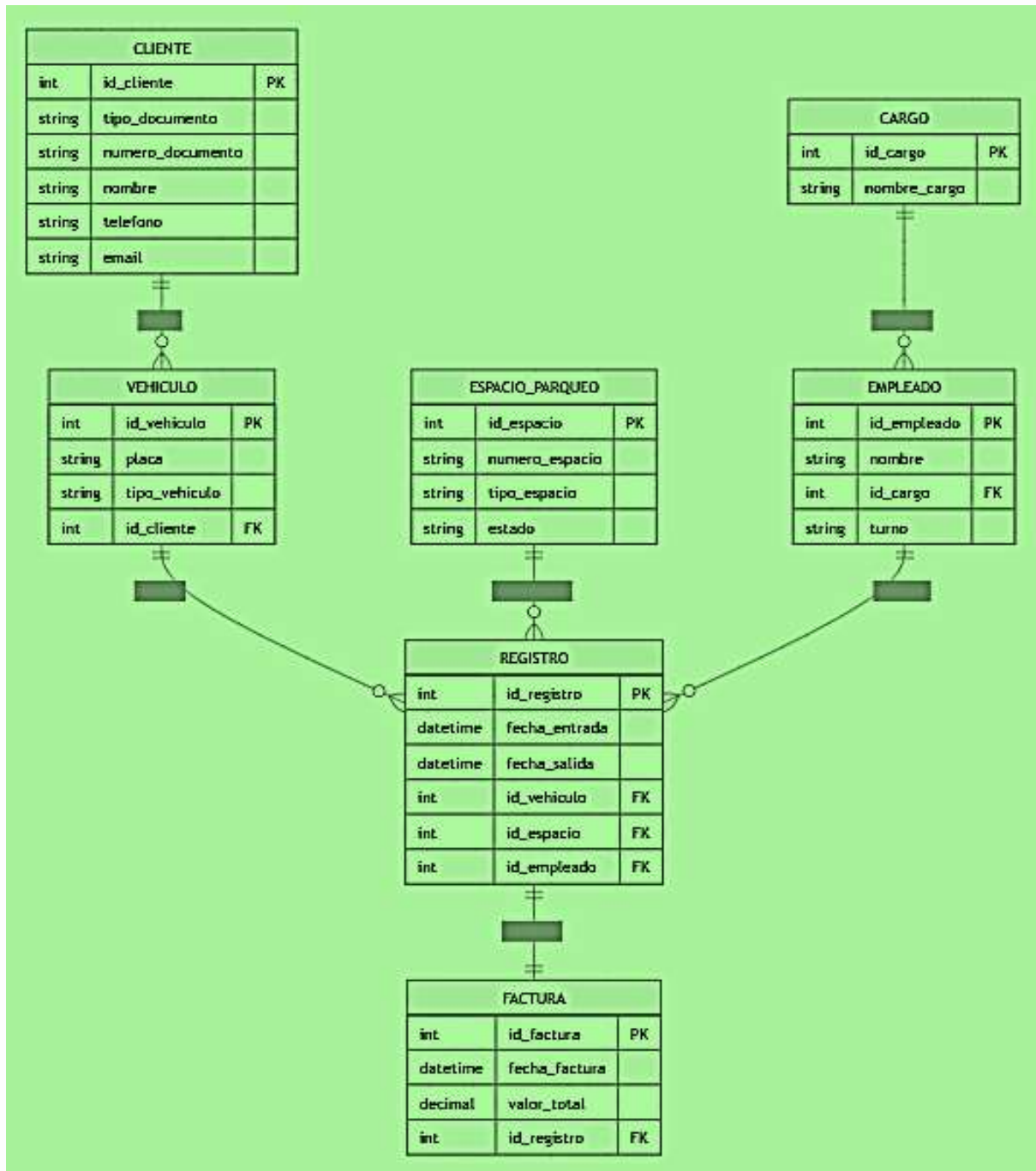
Espacio 1:N Registro

Empleado 1:N Registro

Cargo 1:N Empleado

Registro 1:1 Factura

### 4.3 Diagrama Entidad – Relación



Se ha implementado una relación 1:1 estricta entre REGISTRO y FACTURA mediante una restricción UNIQUE en la clave foránea id\_registro, asegurando que no existan cobros duplicados para una misma estancia

## **5. NORMALIZACIÓN**

### **5.1 Primera Forma Normal (1FN)**

Condición:

No existen atributos multivaluados ni grupos repetitivos.

Ejemplo incorrecto:

Cliente (id, nombre, teléfonos múltiples)

Solución:

Un solo teléfono por registro o tabla adicional si fueran múltiples.

Se cumple 1FN porque todos los atributos son atómicos.

### **5.2 Segunda Forma Normal (2FN)**

Condición:

No existen dependencias parciales respecto a una clave compuesta.

Como todas las tablas usan claves primarias simples (id), no existen dependencias parciales.

Se cumple 2FN.

### **5.3 Tercera Forma Normal (3FN)**

Condición:

No existen dependencias transitivas.

Ejemplo que se evitó:

Empleado  $\rightarrow$  Cargo  $\rightarrow$  Descripción del cargo

Si guardáramos nombre\_cargo directamente en Empleado, existiría dependencia transitiva.

Solución aplicada:

Se creó la tabla Cargo independiente.

Se cumple 3FN estricta.



## 6. MODELO RELACIONAL FINAL

- **CLIENTE** (id\_cliente PK, tipo\_documento, numero\_documento, nombre, telefono, email)
- **VEHICULO** (id\_vehiculo PK, placa UNIQUE, tipo\_vehiculo, id\_cliente FK)
- **ESPACIO\_PARQUEO** (id\_espacio PK, numero\_espacio UNIQUE, tipo\_espacio, estado)
- **CARGO** (id\_cargo PK, nombre\_cargo)
- **EMPLEADO** (id\_empleado PK, nombre, turno, id\_cargo FK)
- **REGISTRO** (id\_registro PK, fecha\_entrada, fecha\_salida, id\_vehiculo FK, id\_espacio FK, id\_empleado FK)
- **FACTURA** (id\_factura PK, fecha\_factura, valor\_total, id\_registro FK UNIQUE)
- **TARIFA** (id\_tarifa PK, tipo\_vehiculo UNIQUE, valor\_hora)

Restricciones aplicadas:

- Claves primarias
- Claves foráneas
- Integridad referencial
- Restricción UNIQUE en placa
- Restricción UNIQUE en id\_registro en FACTURA (garantiza 1:1)

## 7. DEPENDENCIAS FUNCIONALES FORMALES

### 7.1 Definición

Una dependencia funcional se expresa como:

$$X \rightarrow Y$$

Lo que significa que el conjunto de atributos X determina de manera única el conjunto de atributos Y.

En términos prácticos:

Si conozco el valor de X, puedo conocer exactamente el valor de Y.

### 7.2 Dependencias Funcionales por Tabla

#### CLIENTE

Atributos:

(id\_cliente, tipo\_documento, numero\_documento, nombre, telefono, email)

Dependencias funcionales:

id\_cliente  $\rightarrow$  tipo\_documento, numero\_documento, nombre, telefono, email

Además:

numero\_documento  $\rightarrow$  id\_cliente, tipo\_documento, nombre, telefono, email

(si numero\_documento es único en el sistema)

Clave candidata:

- id\_cliente
- numero\_documento (si se define UNIQUE)

## **VEHICULO**

Atributos:

(id\_vehiculo, placa, tipo\_vehiculo, id\_cliente)

Dependencias:

id\_vehiculo  $\rightarrow$  placa, tipo\_vehiculo, id\_cliente

placa  $\rightarrow$  id\_vehiculo, tipo\_vehiculo, id\_cliente

Clave candidata:

- id\_vehiculo
- placa (UNIQUE)

No existen dependencias transitivas.

## **ESPACIO\_PARQUEO**

Atributos:

(id\_espacio, numero\_espacio, tipo\_espacio, estado)

Dependencias:

id\_espacio  $\rightarrow$  numero\_espacio, tipo\_espacio, estado

numero\_espacio  $\rightarrow$  id\_espacio, tipo\_espacio, estado

Clave candidata:

- id\_espacio
- numero\_espacio (UNIQUE)

## **CARGO**

Atributos:

(id\_cargo, nombre\_cargo)

Dependencia:

$\text{id\_cargo} \rightarrow \text{nombre\_cargo}$

Si nombre\_cargo es único:

$\text{nombre\_cargo} \rightarrow \text{id\_cargo}$

## **EMPLEADO**

Atributos:

(id\_empleado, nombre, turno, id\_cargo)

Dependencia:

$\text{id\_empleado} \rightarrow \text{nombre, turno, id\_cargo}$

No existe:

$\text{id\_empleado} \rightarrow \text{nombre\_cargo}$

porque nombre\_cargo depende de id\_cargo.

Esto elimina dependencia transitiva.

Si no existiera tabla CARGO:

$\text{id\_empleado} \rightarrow \text{id\_cargo}$

$\text{id\_cargo} \rightarrow \text{nombre\_cargo}$

Entonces:

$\text{id\_empleado} \rightarrow \text{nombre\_cargo}$  (dependencia transitiva)

Por eso se creó CARGO.

## **REGISTRO**

Atributos:

(id\_registro, fecha\_entrada, fecha\_salida, id\_vehiculo, id\_espacio, id\_empleado)

Dependencia:

$\text{id\_registro} \rightarrow \text{fecha\_entrada}, \text{fecha\_salida}, \text{id\_vehiculo}, \text{id\_espacio}, \text{id\_empleado}$

No existen claves compuestas.

No existen dependencias parciales.

Cumple 2FN y 3FN.

## **FACTURA**

Atributos:

( $\text{id\_factura}, \text{fecha\_factura}, \text{valor\_total}, \text{id\_registro}$ )

Dependencias:

$\text{id\_factura} \rightarrow \text{fecha\_factura}, \text{valor\_total}, \text{id\_registro}$

Si  $\text{id\_registro}$  es UNIQUE:

$\text{id\_registro} \rightarrow \text{id\_factura}, \text{fecha\_factura}, \text{valor\_total}$

Esto garantiza relación 1:1 entre Registro y Factura.

### **7.3 Verificación Formal de 3FN**

Una relación está en Tercera Forma Normal si:

1. Está en 2FN
2. No existen dependencias transitivas
3. Para toda dependencia  $X \rightarrow Y$ , se cumple:
  - X es superclave
  - o Y es atributo primo

Verificación:

En todas las tablas:

- El determinante es clave primaria.
- No existen atributos no clave que determinen otros atributos no clave.
- No existen dependencias parciales.

Conclusión formal:

El modelo cumple estrictamente Tercera Forma Normal (3FN).

## 8. SCRIPT SQL – SISTEMA DE PARQUEADERO

```
-- =====  
  
-- 1. TABLA TARIFA  
  
-- =====  
  
CREATE TABLE tarifa (  
    id_tarifa SERIAL PRIMARY KEY,  
    tipo_vehiculo VARCHAR(20) NOT NULL UNIQUE,  
    valor_hora DECIMAL(10,2) NOT NULL  
);  
  
INSERT INTO tarifa (tipo_vehiculo, valor_hora)  
VALUES ('Carro', 5000), ('Moto', 2000), ('Bicicleta', 1000);  
  
-- =====  
  
-- 2. TABLA CLIENTE  
  
-- =====  
  
CREATE TABLE cliente (  
    id_cliente SERIAL PRIMARY KEY,  
    tipo_documento VARCHAR(20) NOT NULL,  
    numero_documento VARCHAR(20) NOT NULL UNIQUE,  
    nombre VARCHAR(100) NOT NULL,  
    telefono VARCHAR(15),  
    email VARCHAR(100)  
);
```

```
-- =====
```

```
-- 3. TABLA VEHICULO
```

```
-- =====
```

```
CREATE TABLE vehiculo (
    id_vehiculo SERIAL PRIMARY KEY,
    placa VARCHAR(10) NOT NULL UNIQUE,
    tipo_vehiculo VARCHAR(20) NOT NULL,
    id_cliente INT NOT NULL,
    CONSTRAINT fk_cliente FOREIGN KEY (id_cliente) REFERENCES
cliente(id_cliente)
);
```

```
-- =====
```

```
-- 4. TABLA ESPACIO_PARQUEO
```

```
-- =====
```

```
CREATE TABLE espacio_parqueo (
    id_espacio SERIAL PRIMARY KEY,
    numero_espacio VARCHAR(10) NOT NULL UNIQUE,
    tipo_espacio VARCHAR(20) NOT NULL,
    estado VARCHAR(20) DEFAULT 'Disponible' CHECK (estado IN ('Disponible',
'Ocupado', 'Mantenimiento'))
);
```



```
-- =====
```

```
-- 5. TABLA CARGO
```

```
-- =====
```

```
CREATE TABLE cargo (
```

```
    id_cargo SERIAL PRIMARY KEY,
```

```
    nombre_cargo VARCHAR(50) NOT NULL UNIQUE
```

```
);
```

```
-- =====
```

```
-- 6. TABLA EMPLEADO
```

```
-- =====
```

```
CREATE TABLE empleado (
```

```
    id_empleado SERIAL PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    turno VARCHAR(20),
```

```
    id_cargo INT NOT NULL,
```

```
    CONSTRAINT fk_cargo FOREIGN KEY (id_cargo) REFERENCES cargo(id_cargo)
```

```
);
```

```
-- =====
```

```
-- 7. TABLA REGISTRO
```

```
-- =====
```

```
CREATE TABLE registro (
```

```
    id_registro SERIAL PRIMARY KEY,
```

```

    fecha_entrada TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    fecha_salida TIMESTAMP,
    id_vehiculo INT NOT NULL,
    id_espacio INT NOT NULL,
    id_empleado INT NOT NULL,
    CONSTRAINT fk_vehiculo FOREIGN KEY (id_vehiculo) REFERENCES
vehiculo(id_vehiculo),
    CONSTRAINT fk_espacio FOREIGN KEY (id_espacio) REFERENCES
espacio_parking(id_espacio),
    CONSTRAINT fk_empleado FOREIGN KEY (id_empleado) REFERENCES
empleado(id_empleado)
);

```

```
-- =====
```

```
-- 8. TABLA FACTURA
```

```
-- =====
```

```

CREATE TABLE factura (
    id_factura SERIAL PRIMARY KEY,
    fecha_factura TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    valor_total DECIMAL(10,2) NOT NULL,
    id_registro INT NOT NULL UNIQUE,
    CONSTRAINT fk_registro FOREIGN KEY (id_registro) REFERENCES
registro(id_registro)
);

```

## 9. COMPONENTES AVANZADOS DE BASE DE DATOS

### 9.1 Índices (Optimización)

-- Índices para optimización de consultas

```
CREATE INDEX idx_vehiculo_placa ON vehiculo(placa);
```

```
CREATE INDEX idx_registro_fecha_entrada ON registro(fecha_entrada);
```

```
CREATE INDEX idx_registro_vehiculo ON registro(id_vehiculo);
```

```
CREATE INDEX idx_factura_fecha ON factura(fecha_factura);
```

Los índices mejoran el rendimiento en búsquedas frecuentes como:

- Consulta por placa
- Reportes por fecha
- Facturación por rango de tiempo

### 9.2 Vista (View) Para Reportes

```
CREATE VIEW vista_registros_completos AS
```

```
SELECT
```

```
    r.id_registro,
```

```
    c.nombre AS cliente,
```

```
    v.placa,
```

```
    e.numero_espacio,
```

```
    emp.nombre AS empleado,
```

```
    r.fecha_entrada,
```

```
    r.fecha_salida
```

```
FROM registro r
```

```
JOIN vehiculo v ON r.id_vehiculo = v.id_vehiculo
```

JOIN cliente c ON v.id\_cliente = c.id\_cliente

JOIN espacio\_parking e ON r.id\_espacio = e.id\_espacio

JOIN empleado emp ON r.id\_empleado = emp.id\_empleado;

Permite generar reportes sin afectar la estructura interna de las tablas.

### 9.3 Función Para Calcular Valor A Pagar

CREATE OR REPLACE FUNCTION calcular\_valor(

p\_id\_vehiculo INT,

hora\_entrada TIMESTAMP,

hora\_salida TIMESTAMP

) RETURNS DECIMAL AS \$\$

DECLARE

v\_tipo VARCHAR(20);

v\_tarifa DECIMAL;

horas INTEGER;

BEGIN

-- Obtiene el tipo de vehículo para saber qué tarifa aplicar

SELECT tipo\_vehiculo INTO v\_tipo FROM vehiculo WHERE id\_vehiculo =  
p\_id\_vehiculo;

-- Busca el precio actual en la tabla de tarifas

SELECT valor\_hora INTO v\_tarifa FROM tarifa WHERE tipo\_vehiculo = v\_tipo;

-- Calcula horas (redondeando hacia arriba)

horas := CEIL(EXTRACT(EPOCH FROM (hora\_salida - hora\_entrada)) / 3600);

IF horas <= 0 THEN horas := 1; END IF;

```

    RETURN horas * v_tarifa;

END;

$$ LANGUAGE plpgsql;

```

#### 9.4 Triggers Automáticos

##### **TRIGGER 1 – Cambiar estado del espacio a “Ocupado”**

```

CREATE OR REPLACE FUNCTION ocupar_espacio()

RETURNS TRIGGER AS $$

BEGIN

    UPDATE espacio_parqueo

    SET estado = 'Ocupado'

    WHERE id_espacio = NEW.id_espacio;

    RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_ocupar_espacio

AFTER INSERT ON registro

FOR EACH ROW

EXECUTE FUNCTION ocupar_espacio();

```

##### **TRIGGER 2 – Liberar espacio cuando se registra salida**

```

CREATE OR REPLACE FUNCTION liberar_espacio()

RETURNS TRIGGER AS $$

BEGIN

```

```

IF NEW.fecha_salida IS NOT NULL THEN

    UPDATE espacio_parqueo

    SET estado = 'Disponible'

    WHERE id_espacio = NEW.id_espacio;

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_liberar_espacio

AFTER UPDATE ON registro

FOR EACH ROW

EXECUTE FUNCTION liberar_espacio();

```

### **TRIGGER 3 – Validar que el espacio no esté ocupado antes de entrar**

```

CREATE OR REPLACE FUNCTION validar_disponibilidad()

RETURNS TRIGGER AS $$

BEGIN

    IF (SELECT estado FROM espacio_parqueo WHERE id_espacio = NEW.id_espacio) !=

'Disponible' THEN

        RAISE EXCEPTION 'Error: El espacio seleccionado ya se encuentra ocupado o en

mantenimiento.';

    END IF;

    RETURN NEW;

END;

```

```
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_validar_disponibilidad  
BEFORE INSERT ON registro  
FOR EACH ROW  
EXECUTE FUNCTION validar_disponibilidad();
```

Los triggers permiten automatizar reglas de negocio directamente en la base de datos, garantizando consistencia incluso si la aplicación falla o es modificada.

Esto asegura que un espacio no permanezca marcado como ocupado después de que un vehículo se retire.

## **10. CONCLUSIONES**

El modelo diseñado cumple con los principios del modelo entidad–relación y las reglas de normalización hasta Tercera Forma Normal (3FN).

Se eliminaron redundancias, se evitaron dependencias transitivas y se garantizó integridad referencial.

El diseño es escalable, consistente y apto para implementación en motores de base de datos relacionales como MySQL, PostgreSQL o SQL Server.